

Transaction Processing

1. What is a Transaction?

A **transaction** is a logical unit of work — one or more database operations (read, insert, delete, modify) that must be treated as a single, indivisible whole.

```
BEGIN TRANSACTION
... read/write operations ...
END TRANSACTION
```

Granularity — the size of the data item a transaction's operations act on: from a **field**, to a **record**, up to a whole **disk block**. The concepts of transaction processing don't depend on which granularity is chosen — the same reasoning applies regardless.

Key terms

Term	Meaning
Buffer	Temporary in-memory storage for blocks being read/written
Read-set	All data items a transaction reads
Write-set	All data items a transaction writes
Buffer replacement policy	Determines which buffered block gets evicted when the buffer is full

Interleaved execution: a single CPU can only run one process at a time, but rapidly alternating between transactions (A executes a few operations, then B, then back to A...) creates the *illusion* of simultaneous execution — this interleaved model is the foundation almost all concurrency-control theory (see the next topic) is built around, since it's what actually happens on real hardware.

2. Why Concurrency Control Is Needed — Four Concrete Problems

Without any coordination, concurrent transactions accessing the same data can produce genuinely incorrect results:

Lost Update

Two transactions read the same item, then each writes back a value computed from that read — one update silently **overwrites** the other, as if it never happened.

```
T1: read(X); X = X + N      T2: read(X); X = X - M
```

If both read **X** before either writes it back, whichever writes **last** wins — the other transaction's update is completely lost, with no error raised at all.

Temporary Update (Dirty Read)

A transaction reads data that was written by **another transaction that later fails/aborts** — meaning it read a value that, in the end, never actually should have existed.

```
T1 writes X, then FAILS before committing → X must be rolled back to its old value
T2 already read the "dirty" (soon-to-be-rolled-back) value of X in the meantime
```

T2 now has a value that's invalid — the system must restore **X**, but **T2** has already acted on the wrong data.

Incorrect Summary

One transaction computes an **aggregate** (sum, count, average) while another transaction is concurrently updating some of the individual items being aggregated — the aggregate ends up counting a state that never actually existed at any single point in time.

```
T1: transfers N seats from Flight X to Flight Y
T2: sums seats across ALL flights, concurrently
```

If **T2** reads **X** **after** **T1** subtracted **N**, but reads **Y** **before** **T1** added **N** back, the total is short by **N** — an inconsistent snapshot that was never real.

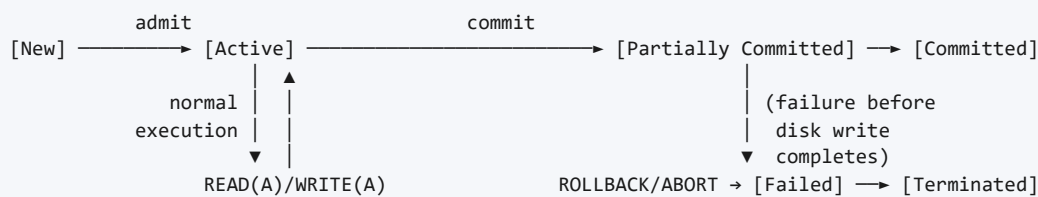
Unrepeatable Read

A transaction reads the **same** data item **twice**, and gets **two different values**, because another transaction updated it in between the two reads.

Problem	Cause	Consequence
Lost Update	One update overwrites another	Final value is wrong
Dirty Read	Reads data from a failed transaction	Uses invalid, never-real data
Incorrect Summary	Aggregate computed mid-update	Total is inaccurate
Unrepeatable Read	Same item read twice, different results	Inconsistent view within one transaction

Interview soundbite: "All four problems have the same root cause — uncontrolled interleaving lets one transaction observe or build on a partial, in-flux state of the database that no serial execution could ever have produced. Concurrency control techniques (locking, timestamp ordering, MVCC — covered in the next topic) all exist specifically to eliminate this root cause."

3. Transaction States (Lifecycle)



State	Meaning
Active	Executing normally (reads/writes in progress)
Partially Committed	Finished all statements, but changes not yet guaranteed permanently on disk
Committed	All changes permanently recorded — durable, even across a crash
Failed	Something went wrong; normal execution cannot continue
Terminated	Fully done (either via commit, or rollback/abort completing)

Why "Partially Committed" exists as its own state: a transaction can finish executing all its logical steps, yet a crash could still occur *before* its changes are safely written to disk — this state exists precisely to capture that narrow, dangerous window, which is exactly why the **system log** (below) matters so much.

4. The System Log — the foundation of recovery

The **system log** is a special, **sequential, append-only** file where the DBMS records everything needed to recover from a crash.

- Recently-written log records are typically kept in a memory **buffer**, periodically flushed to disk — the log itself is also periodically backed up to external storage, protecting against everything

except catastrophic media failure.

Log record types

Record	Meaning
[start, T]	Transaction T has started
[write, T, X, old_value, new_value]	T wrote item X
[read, T, X]	T read item X
[commit, T]	T finished successfully — changes are now permanent
[abort, T]	T failed/was rolled back — its changes must be undone

Recovery using Undo and Redo

- **Undo** — for a transaction that **did not commit** before a crash: reverse its changes, using the log's `old_value` entries.
- **Redo** — for a transaction that **did commit**, but whose changes weren't yet flushed from buffer to disk before the crash: reapply its changes, using the log's `new_value` entries.

Crash → read the log → UNDO every uncommitted transaction's writes
→ REDO every committed transaction's writes (if not yet on disk)

The Commit Point — a critical rule

Before a transaction is allowed to commit, its log records must be force-written to disk (even though data blocks themselves might still be safely deferred).

Why: log entries are often kept in memory momentarily for efficiency — if the system crashed *before* those log entries reached disk, there'd be no record at all that the transaction ever ran, making its changes (which might already be partially on disk) unrecoverable/inconsistent. Force-writing the log at commit time guarantees that once a transaction is told "you're committed," its outcome is **permanently, durably recorded** no matter what happens immediately afterward — this is exactly the mechanism that delivers the **Durability** ACID property (below).

5. ACID Properties

Property	Meaning	How it's enforced
Atomicity	Either the transaction completes fully, or it has no effect at all	If a transaction fails mid-way, the recovery subsystem undoes any partial changes
Consistency	A transaction moves the database from one valid state to another valid state	Integrity constraints (from the Relational Model topic) are checked; application logic respects business rules
Isolation	Concurrent transactions don't interfere with each other's intermediate states	Concurrency control mechanisms (next topic) — each transaction behaves as if it ran alone
Durability	Once committed, changes survive any subsequent failure	The recovery subsystem, via the write-ahead log and the commit-point rule above

Worked example (banking): if two people withdraw from the same account concurrently, **isolation** ensures each withdrawal is processed as if it happened in some definite order (not interleaved incorrectly, as in the lost-update problem). Once a withdrawal transaction commits, **durability** guarantees the new balance is permanent — even if the system crashes one millisecond later.

6. Schedules (Histories)

A **schedule (history)** s is the actual chronological sequence of operations from potentially **several** transactions, as they were physically interleaved during execution.

Shorthand notation: $r1(X)$ = "T1 reads X," $w2(Y)$ = "T2 writes Y," etc.

Conflicting operations

Two operations **conflict** if: (1) they belong to **different** transactions, (2) they access the **same** data item, and (3) **at least one of them is a write**.

Case	Conflict?
$r1(X), r2(X)$	No — two reads never conflict
$r1(X), w2(X)$	Yes
$w1(X), w2(X)$	Yes
$r1(X), r1(Y)$ (same transaction)	No — same transaction, order is fixed anyway

Why conflicts matter: two conflicting operations' **relative order** genuinely affects the outcome — swapping them can change the final database state or what a later read observes. Two **non-conflicting** operations can be freely reordered (or executed in either order) without changing the outcome at all — this single idea is the entire foundation of conflict-serializability testing, below.

Committed Projection $C(S)$

Real systems constantly have transactions in flight — $C(S)$ is the version of schedule S containing **only** the operations belonging to transactions that **actually committed**, discarding operations from aborted or still-running transactions. This is what recovery and consistency-checking actually care about — an aborted transaction's operations should never be considered part of "what really happened" to the database.

Schedule Classification by Recoverability

Type	Rule
Recoverable	If T_j reads a value written by T_i , then T_i must commit before T_j commits
Cascadeless	T_j may only read a value written by T_i after T_i has already committed (stronger — avoids the <i>need</i> to cascade an abort)
Strict	T_j may not even read or write an item that T_i has written, until T_i commits or aborts (strongest)

Why does this hierarchy exist? A non-recoverable schedule can leave the database in a state that's *impossible to fix* if T_i later aborts — T_j might have already committed based on a value that should never have existed. A recoverable-but-not-cascadeless schedule avoids that catastrophe but can still force a painful **cascading rollback** (if T_i aborts, every transaction that read T_i 's uncommitted write must *also* be rolled back, potentially triggering a further chain). Strict schedules eliminate even the possibility of that cascade by never letting anyone touch an item until its writer has definitively finished.

7. Serializability

A schedule is serializable if its final effect on the database is identical to *some* serial (one-transaction-at-a-time) execution of the same transactions.

- **Serial schedule** — transactions execute one completely after another, with zero interleaving. Always trivially "correct" (assuming each individual transaction is correct), but offers no concurrency/performance benefit at all.
- **Nonserial schedule** — operations from multiple transactions are interleaved (for performance).
- **The goal of serializability theory:** allow the performance benefits of interleaving, while still guaranteeing the *same* correctness a serial execution would give.

Conflict Serializability

A schedule is **conflict-serializable** if it can be transformed into a serial schedule by repeatedly swapping **non-conflicting**, adjacent operations.

Worked example: if a schedule's only cross-transaction conflicts are, say, $w1(X)$ occurring before $r2(X)$ and $w1(X)$ before $w2(X)$ — meaning every conflict requires $T1$'s operation to come before $T2$'s — then any operations that *don't* conflict (e.g., $T1$'s operations on a completely different item Y) can be freely reordered around them without changing the outcome. This means the schedule can always be rearranged into the equivalent serial order $T1 \rightarrow T2$ — hence conflict-serializable.

Testing Conflict Serializability — the Precedence Graph

1. Create one node per transaction.
2. For every pair of **conflicting** operations (O_i from T_i , O_j from T_j) where O_i occurs **before** O_j , draw a directed edge $T_i \rightarrow T_j$.
3. **The schedule is conflict-serializable if and only if this graph has no cycle.**
4. If acyclic, any **topological sort** of the graph gives a valid equivalent serial order.

Example: edges $T1 \rightarrow T2$, $T2 \rightarrow T3$, $T3 \rightarrow T1$ → CYCLE → NOT conflict-serializable

Example: edges $T1 \rightarrow T2$, $T1 \rightarrow T3$ → no cycle → serializable, e.g., order $T1, T2, T3$ or $T1, T3, T2$

Why DBMSs Don't Actually *Test* for Serializability at Runtime

Building the actual precedence graph requires knowing the **complete** schedule — but a real DBMS interleaves operations dynamically, live, without knowing the future. Worse, if you discovered *after the fact* that a schedule wasn't serializable, you'd have to **undo everything** already executed — catastrophically expensive.

What DBMSs actually do instead: enforce **protocols** (rules followed *during* execution, like locking schemes or timestamp ordering — the entire subject of the next topic) that **guarantee** every schedule they ever produce is serializable **by construction**, without ever needing to test after the fact.

In practice, since only committed work ultimately matters, a schedule is considered serializable if its **committed projection** $C(S)$ is serializable — this lets protocols not worry about transactions that end up aborting anyway.

View Serializability (a broader, harder-to-check alternative)

Two schedules are **view equivalent** if: (1) each transaction reads the same values from the same sources in both, and (2) the same transaction performs the final write on each item in both.

- **Every conflict-serializable schedule is also view-serializable** — but the reverse isn't true. View serializability is a strictly **broader** (less restrictive) definition of "equivalent to some serial schedule."
- The gap between them only actually matters when **blind writes** occur (a transaction writes an item **without** having read it first) — under the common "no blind writes" assumption, conflict serializability and view serializability coincide exactly.
- **Practical trade-off:** view serializability is provably **harder to test** (no simple graph-based test exists), which is exactly why real systems rely on the stricter, more conservative, but efficiently-checkable conflict serializability instead.

Beyond Serializability — relaxed schemes

Serializability is sometimes **stricter than necessary** — some non-serializable schedules are still perfectly *correct* for specific applications, if you understand the semantics of the operations involved (e.g., bank deposits/withdrawals that are individually commutative in their effect on the final balance, even if interleaved in a way that isn't strictly serializable). Recognizing this, researchers developed **relaxed concurrency control schemes** that permit certain safe, semantically-justified interleavings serializability would otherwise needlessly forbid — trading some theoretical strictness for better real-world concurrency in specific, well-understood cases.

Interview Questions With Answers

Q1. Walk through exactly how the lost update problem occurs, step by step.

Answer: Two transactions, T1 and T2, both read the same data item X into their own local working copies before either writes anything back. Each computes a new value based on its own reading of X

(e.g., T1 computes $X+N$, T2 computes $X-M$) and writes its result back. Whichever transaction's write happens *last* simply overwrites the other's result entirely — the earlier write's effect is completely erased, as if that transaction had never run at all, with no error or conflict detected by the system.

Q2. What's the difference between a dirty read and an unrepeatable read?

Answer: A dirty read occurs when a transaction reads a value written by another transaction that has **not yet committed** (and might later abort, making that value entirely invalid/never-real). An unrepeatable read occurs when a transaction reads the **same** item twice within its own execution and gets two **different** values, because another transaction **committed** an update to that item in between the two reads — the value read was always valid at the time it was read, but it's inconsistent across the two reads within one transaction's lifetime.

Q3. Why does the "Partially Committed" transaction state need to exist as a distinct state, separate from both "Active" and "Committed"?

Answer: A transaction can finish executing every one of its logical operations (making it no longer "Active" in the sense of doing more work) while its changes are still only reflected in memory buffers, not yet guaranteed to be durably written to disk. If a crash occurs during exactly this narrow window, the transaction's fate depends entirely on what the recovery subsystem's log shows — it needs its own distinct state to capture this specific "done executing, but durability not yet guaranteed" situation, distinct from being fully committed (durability guaranteed) or still actively running.

Q4. Why must a transaction's log records be force-written to disk before it's allowed to commit, even if the actual data blocks can be written later?

Answer: The commit point is the moment the system promises a transaction's effects are permanent, no matter what happens next — including an immediate crash. If the log records (which are what recovery uses to redo committed work not yet reflected in the data files) were still sitting only in a memory buffer at that moment, a crash immediately after "commit" was declared could lose all record that the transaction ever happened, even though the commit promise had already been made to whoever/whatever depended on it. Force-writing the log (not necessarily the data blocks themselves) at commit time is the minimum guarantee needed to make that durability promise actually hold — the data blocks can be brought up to date later via redo, using exactly this log.

Q5. Explain the difference between a recoverable, a cascadeless, and a strict schedule, and why each is progressively stronger.

Answer: A recoverable schedule only requires that if Tj reads something Ti wrote, Ti must commit before Tj does — this prevents the worst outcome (Tj committing based on a value that later turns out to have never legitimately existed) but still permits Tj to read Ti's uncommitted write while Ti is still active, meaning if Ti later aborts, Tj (and anyone who read from Tj, and so on) must also be rolled back in a cascade. A cascadeless schedule closes this gap by only allowing Tj to read a value from Ti *after* Ti has already committed, eliminating the need for cascading rollbacks entirely. A strict schedule goes further still, preventing Tj from even reading *or writing* any item Ti has touched until

T_i has committed or aborted — this is the strongest of the three and is what most real systems actually implement, since it simplifies undo/redo logic during recovery the most.

Q6. How does the precedence graph test for conflict serializability work, and why does an acyclic graph guarantee an equivalent serial schedule exists?

Answer: You build one node per transaction, then draw a directed edge $T_i \rightarrow T_j$ for every pair of conflicting operations where T_i 's operation occurred before T_j 's in the actual schedule. If this graph contains a cycle, it means there's a group of transactions that would each need to come both "before" and "after" each other simultaneously in any serial ordering — an impossible contradiction, meaning no equivalent serial order can exist, so the schedule is not conflict-serializable. If the graph is acyclic, a topological sort of it gives a valid ordering where every conflict-derived precedence requirement (T_i before T_j) is satisfied — that topological order is exactly the equivalent serial schedule.

Q7. Why don't real DBMSs actually build a precedence graph and test schedules for serializability while they're running?

Answer: Testing serializability requires knowing the complete schedule (every operation from every transaction) in advance, but a live DBMS interleaves operations dynamically as transactions arrive and execute — it cannot know the future shape of the schedule while deciding what to do right now. Even if it somehow could test retroactively, discovering after the fact that an already-executed schedule wasn't serializable would require undoing everything already done — a prohibitively expensive, essentially unworkable approach. Instead, DBMSs enforce concurrency control protocols (locking, timestamp ordering, etc.) that guarantee, by construction, that any schedule they could possibly produce is always serializable — avoiding the need to test at all.

Q8. Why is view serializability considered "broader" than conflict serializability, and why do real systems still prefer the stricter conflict serializability?

Answer: View serializability only requires that each transaction read the same values from the same sources, and that the same transaction perform the final write on each item, across two schedules being compared — it doesn't require preserving the exact relative order of every conflicting operation the way conflict serializability does. This means some schedules are view-serializable without being conflict-serializable (specifically when "blind writes" occur — writes not preceded by a read of that same item) — so view serializability accepts a strictly larger set of schedules as "correct." Real systems still prefer conflict serializability because it has a simple, efficient graph-based test (build the precedence graph, check for cycles), while no comparably efficient test exists for view serializability in general — practical enforceability wins out over accepting a theoretically larger set of correct schedules.

Q9. Scenario: A banking system processes many concurrent deposit and withdrawal transactions on different accounts, and a DBA notices the actual execution schedule is technically not conflict-serializable, yet the final account balances are all correct

and consistent. How is this possible, and what does it suggest about the concurrency control being used?

Answer: This is possible because serializability, while sufficient for correctness, is not always *necessary* — some non-serializable interleavings are still safe if you understand the semantics of the specific operations involved. Deposits and withdrawals to the *same* account are typically commutative in their net effect on the final balance (adding N then subtracting M gives the same final result regardless of the exact interleaving of the underlying read-modify-write steps, as long as each individual increment/decrement is applied atomically) — so an interleaving that superficially violates strict conflict-serializability at the operation level can still produce a database state indistinguishable from some valid serial execution's *result*, even if it doesn't match the stricter operation-by-operation conflict-serializability test. This suggests the system may be using a relaxed concurrency control scheme deliberately designed to exploit this specific semantic property (e.g., treating deposits/withdrawals as atomic incremental operations rather than generic read-then-write operations), trading strict serializability's guarantees for better real-world concurrency in a case where the relaxation is provably still safe.